

Problem Statement Document

Distributed AI System Designs using PySpark RDDs and Intelligent Agents
Five Problem Domains: Recommendation • Disaster Detection • Fraud Detection • Supply Chain • Learning Analytics

#	System	Core Distributed-Computing Focus
1	Hybrid AI Recommendation Engine Design	Design a hybrid AI-based recommendation system that integrates content-based a...
2	Smart Disaster Detection System	Develop a distributed AI system using PySpark and RDD operations to detect nat...
3	AI-Based Fraud Detection Framework	Design a scalable fraud detection system using PySpark RDDs to process large v...
4	Autonomous Supply Chain Optimization System	Construct a distributed AI system using PySpark for optimizing supply chain op...
5	Personalized Learning Analytics Platform	Design an AI-driven educational platform that uses PySpark and RDDs to analyze...

Problem 1: Hybrid AI Recommendation Engine Design

Background & Motivation

E-commerce, streaming, and content platforms must recommend relevant items to millions of users from catalogs containing thousands to millions of items. Purely collaborative approaches suffer from the cold-start problem for new users/items, while purely content-based approaches fail to capture the wisdom of crowds. A single-machine solution cannot scale to the volume of user-item interactions generated by a large platform, and static recommendation logic cannot adapt to rapidly changing user intent.

Problem Statement

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

Key Objectives

- Ingest and process large-scale, sparse user-item rating/interaction data using PySpark RDDs.
- Implement collaborative filtering via distributed item-item / user-user similarity computation.
- Implement content-based filtering using item metadata / feature profiles.
- Fuse both signals into a single hybrid ranking score to mitigate cold-start and sparsity issues.
- Introduce an intelligent agent layer that consumes real-time feedback (clicks, ratings, dwell time) to adaptively re-weight the hybrid blend without retraining the entire model.

Constraints & Requirements

- Must scale horizontally across a Spark cluster as the number of users/items grows.
- Recommendations should update within acceptable latency for near real-time personalization.
- System must remain resilient to missing/sparse rating data (cold-start scenarios).

Problem 2: Smart Disaster Detection System

Background & Motivation

Natural disasters such as floods and earthquakes generate heterogeneous, high-velocity data from satellite imagery, IoT/ground sensors, and social media simultaneously. Relying on a single data source delays detection and increases false negatives, while manual correlation of multi-source data is too slow for life-critical early warning.

Problem Statement

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

Key Objectives

- Continuously ingest multi-source streaming data (sensor telemetry, satellite-derived anomaly indices, and social-media disaster-keyword mentions) using distributed RDD pipelines.
- Aggregate and normalize heterogeneous signals per geographic zone using RDD transformations (map, reduceByKey, join).
- Apply a data-fusion function to compute a unified, weighted disaster-risk index per zone.
- Deploy coordinating intelligent agents that evaluate the fused risk index against adaptive thresholds and trigger early-warning alerts and response strategies.

Constraints & Requirements

- Must process data from at least three heterogeneous sources concurrently.
- Detection latency must be minimized to support life-saving early warnings.
- False-alarm rate must be controllable via adaptive agent thresholds.

Problem 3: AI-Based Fraud Detection Framework

Background & Motivation

Financial institutions process millions of transactions daily. Fraudulent transactions are rare, evolve quickly, and must be detected within seconds to prevent loss. Centralized, batch-only analysis cannot keep pace with transaction volume or adversarial adaptation by fraudsters.

Problem Statement

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

Key Objectives

- Distribute transaction ingestion and per-account statistical profiling (mean, standard deviation) using RDD aggregation operations (aggregateByKey).
- Implement a distributed anomaly-scoring function (e.g., z-score deviation combined with behavioral rules such as odd-hour activity).
- Rank and filter suspicious transactions at scale using RDD filter/sort transformations.
- Employ collaborating intelligent agents (detector + analyst-feedback agent) that adapt alert thresholds from continuous human-in-the-loop feedback to improve precision over time.

Constraints & Requirements

- Must scale to high-volume transaction streams without single-node bottlenecks.
- Should minimize false positives while not missing true fraud (precision/recall balance).
- Detection logic must continuously improve using a feedback loop rather than static rules.

Problem 4: Autonomous Supply Chain Optimization System

Background & Motivation

Global supply chains involve many warehouses, thousands of SKUs, fluctuating demand, and distributed logistics costs. Centralized optimization does not scale and cannot react quickly to local shortages or surpluses, leading to stockouts, overstocking, and increased operating cost.

Problem Statement

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

Key Objectives

- Process large-scale inventory and demand-forecast datasets across multiple warehouses and products using distributed RDD joins and aggregations.
- Identify shortage and surplus positions per warehouse-product pair via RDD-based gap analysis.
- Design decentralized per-warehouse intelligent agents that autonomously propose inter-warehouse stock transfers, balancing urgency of need against logistics cost.
- Adapt agent decision confidence using historical delivery-performance feedback.

Constraints & Requirements

- Decision-making must be decentralized (no single point of failure/bottleneck).
- Must balance cost minimization against service-level/urgency requirements.
- Should scale to large numbers of warehouses and SKUs processed in a distributed fashion.

Problem 5: Personalized Learning Analytics Platform

Background & Motivation

Online and blended learning platforms generate large volumes of behavioral and performance data per learner. Providing every learner with a one-size-fits-all curriculum ignores individual pace and mastery differences, and manual personalization does not scale to thousands of concurrent learners.

Problem Statement

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

Key Objectives

- Ingest and process large-scale student interaction logs (scores, time-on-task, topic attempts) using distributed RDD transformations.
- Compute per-student performance aggregates and per-topic difficulty statistics at scale.
- Identify at-risk learners via distributed filtering against performance thresholds.
- Deploy an adaptive tutor agent that personalizes next-content recommendations per learner using real-time engagement feedback (lesson completion vs. drop-off).

Constraints & Requirements

- Must scale to thousands of concurrent learners and interaction events.
- Personalization must update responsively as new performance/engagement data arrives.
- System should surface actionable, explainable analytics for both learners and educators.